

kickstart_lecture1_python-syntax-I-enhanced

April 19, 2024

0.1 Python Kickstart Lecture 1: Syntax - Code Structure, Data Types, Variables, Basic Operations

0.1.1 Code Structure

```
[1]: # Assigning variable
      # Assign an integer value 9 to variable x.
      # Assign an integer value 10 to variable y.
```

```
[2]: y
```

```
[2]: 10
```

```
[3]: # Variable Names: Start with small or capital letter, underscore
      # Define a floating-point variable starting with a capital letter.
      # Define a floating-point variable starting with an underscore.
```

```
[4]: # Variable Names should be descriptive: CamelCase vs snake_case
      CamelCaseVariable = 40
      snake_case_var = 85
```

```
[5]: # Comments: simple and multiline

      # Define an integer using CamelCase notation.
      # Define an integer using snake_case notation.
```

```
[7]: ''' This is a multiline comment
      which extends
      over three lines'''

      # Multiply x and y and display the result.
```

```
[7]: 90
```

```
[14]: # Indentation

      Radius = 7

      if Radius>5:
```

```
Area = 3.14*Radius**2
Area
```

[14]: 153.86

0.1.2 Basic Data Types

```
[18]: # str

str_var = 'This is a string'
str_var
```

[18]: 'This is a string'

```
[20]: another_str_var = str('Hello')
another_str_var
```

[20]: 'Hello'

```
[23]: # int

x = 8
print(x)
```

8

```
[22]: # float

float_var = 8.3345
float_var
```

[22]: 8.3345

```
[24]: # complex
complex_var = .1 + .1j
complex_var
```

[24]: (0.1+0.1j)

```
[26]: # booleans
bool_var = True
bool_var_2 = False
bool_var and bool_var_2
```

[26]: False

```
[ ]: # check types and content
```

0.1.3 Operators

```
[27]: # Simple Operators, e.g., +, -, /, *, **  
x+y
```

[27]: 18

```
[28]: # Multiply x and y and display the result.
```

[28]: 80

```
[29]: x**y
```

[29]: 1073741824

```
[30]: x/y
```

[30]: 0.8

```
[31]: # Assignment Operators, e.g., +=, -=  
  
x
```

[31]: 8

```
[32]: x += 2  
x
```

[32]: 10

```
[33]: # is identical to  
x = x+2
```

```
[36]: y *= 10  
y
```

[36]: 10000

```
[37]: # Comparison operators, e.g., >,<==, !=  
  
x>y
```

[37]: False

```
[38]: x<y
```

[38]: True

```
[39]: x!=y
```

[39]: True

0.1.4 Casting: Converting data types

[42]: *# convert int to float and back*

```
x = float(x)
type(x)
```

[42]: float

[44]: *# Source for errors?*

```
float_var_3 = 9.7
float_turned_int = int(float_var_3)
float_turned_int
```

[44]: 9

[45]: float(float_turned_int)

[45]: 9.0

0.1.5 Complex Data Types: Lists, Ranges, Tuples, Dicts

[46]: *# create list*

```
list_var = [1,3,4,'x','b']
```

[47]: list_var

[47]: [1, 3, 4, 'x', 'b']

[48]: *# access one element*

```
list_var[3]
```

[48]: 'x'

[51]: *# and from the end*

```
list_var[-3]
```

[51]: 4

[54]: *# access multiple elements*

```
list_var[:3]
```

[54]: [1, 3, 4]

```
[60]: # change list element
list_var
```

```
[60]: [1, 3, 4, 'x', 'b']
```

```
[61]: list_var[3] = 'ABC'
list_var
```

```
[61]: [1, 3, 4, 'ABC', 'b']
```

```
[55]: # ranges
range_var = range(10)
```

```
[57]: for i in range_var:
      print(i)
```

```
0
1
2
3
4
5
6
7
8
9
```

```
[58]: for j in range(4,40,2):
      print(j)
```

```
4
6
8
10
12
14
16
18
20
22
24
26
28
30
32
34
36
38
```

```
[59]: # tuples
```

```
a_tuple = (6,7,8)
a_tuple
```

```
[59]: (6, 7, 8)
```

```
[62]: a_tuple[0]
```

```
[62]: 6
```

```
[63]: # cannot change tuple
a_tuple[0] = 1
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[63], line 2
      1 # cannot change tuple
----> 2 a_tuple[0] = 1

TypeError: 'tuple' object does not support item assignment
```

```
[69]: # dictionnaire work with pairs key:value
```

```
a_dict = {'Dog':'Linda', 'Cat':'Doerthe', 'Rabbit':'Hansi', 'Dog1':
↳ 'Daisy', 'years':[2010,2011,2012,2013]}
a_dict
```

```
[69]: {'Dog': 'Linda',
      'Cat': 'Doerthe',
      'Rabbit': 'Hansi',
      'Dog1': 'Daisy',
      'years': [2010, 2011, 2012, 2013]}
```

```
[67]: a_dict['Dog1']
```

```
[67]: 'Daisy'
```

```
[70]: a_dict['years']
```

```
[70]: [2010, 2011, 2012, 2013]
```

0.1.6 Built-in Functions and Methods

```
[71]: # Functions example: lists
```

```
list_var
```

```
[71]: [1, 3, 4, 'ABC', 'b']
```

```
[72]: # Inspect with type and len functions
```

```
type(list_var)
```

```
[72]: list
```

```
[73]: len(list_var)
```

```
[73]: 5
```

```
[ ]: # Join lists with "+" operation
```

```
[74]: list_var_2 = ['c', 'd', 'e', 'a', 'b', 'c']
```

```
[75]: # Display the current state of list_merged.  
# Display the current state of list_merged.
```

```
[75]: [1, 3, 4, 'ABC', 'b', 'c', 'd', 'e', 'a', 'b', 'c']
```

```
[76]: # Method to append list  
# Display the current state of list_merged.  
# Display the current state of list_merged.
```

```
[76]: [1, 3, 4, 'ABC', 'b', 'c', 'd', 'e', 'a', 'b', 'c', 111]
```

```
[77]: # Which methods are available? Use dir()  
  
# Display the current state of list_merged.
```

```
[77]: ['__add__',  
      '__class__',  
      '__class_getitem__',  
      '__contains__',  
      '__delattr__',  
      '__delitem__',  
      '__dir__',  
      '__doc__',  
      '__eq__',  
      '__format__',  
      '__ge__',  
      '__getattr__',  
      '__getattribute__',  
      '__getitem__',  
      '__gt__',  
      '__hash__',  
      '__init__',  
      '__init_subclass__',  
      '__iter__',  
      '__le__',  
      '__len__',  
      '__lt__',  
      '__module__',  
      '__ne__',  
      '__new__',  
      '__next__',  
      '__reduce__',  
      '__reduce_ex__',  
      '__repr__',  
      '__setattr__',  
      '__setitem__',  
      '__sizeof__',  
      '__str__',  
      '__subclasshook__',  
      'clear',  
      'copy',  
      'count',  
      'extend',  
      'index',  
      'insert',  
      'pop',  
      'remove',  
      'reverse',  
      'sort']
```

```

'__getitem__',
'__getstate__',
'__gt__',
'__hash__',
'__iadd__',
'__imul__',
'__init__',
'__init_subclass__',
'__iter__',
'__le__',
'__len__',
'__lt__',
'__mul__',
'__ne__',
'__new__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__reversed__',
'__rmul__',
'__setattr__',
'__setitem__',
'__sizeof__',
'__str__',
'__subclasshook__',
'append',
'clear',
'copy',
'count',
'extend',
'index',
'insert',
'pop',
'remove',
'reverse',
'sort']

```

```
[81]: # Display the current state of list_merged.
```

```
[81]: 2
```

```
[82]: # Display the current state of list_merged.
```

```
[82]: [1, 3, 4, 'ABC', 'b', 'c', 'd', 'e', 'a', 'b', 'c', 111]
```

```
[83]: # Display the current state of list_merged.
```



```
[83]: 'ABC'
```

```
[84]: # Display the current state of list_merged.
```

```
[84]: [1, 3, 4, 'b', 'c', 'd', 'e', 'a', 'b', 'c', 111]
```

```
[86]: # Sort the elements of list_var_2 in ascending order.
```

```
[87]: list_var_2
```

```
[87]: ['a', 'b', 'c', 'c', 'd', 'e']
```

```
[88]: # Define a list of integers.  
# Sort the list_var_3 in ascending order.
```

```
[89]: list_var_3
```

```
[89]: [1, 3, 5]
```

```
[ ]:
```